

見開き楽譜スキャンPDFのブラウザ完結型高精度クロップ・分割アーキテクチャ設計書

序論

オーケストラや室内楽における見開き楽譜のデジタル化は、変則的なスキャンサイズ(A3や不定形)、劣化した紙面の黄ばみ、非対称な余白といった物理的環境に起因するノイズを多く含んでいる。これらの巨大かつ不均一な画像をブラウザ上で完全に処理し、無劣化の300 DPI A4サイズのPDFファイル(Score Optimizer 2.0)として再構築する要件は、フロントエンドアーキテクチャにおける極限のパフォーマンスチューニングと厳格なメモリ管理を要求する。

過去の実装で頻発した、Reactのライフサイクルにおける競合(マウント・アンマウントの繰り返しによるレンダリングの破綻)、巨大画像(A3・600dpi相当など)をメインスレッドで直接処理することによるUIの凍結、DOM上の座標と元画像のピクセル座標の混同によるクロップ位置のズレ、そしてモバイルデバイスにおけるUIコンテナの見切れは、典型的なブラウザベースの画像処理におけるアンチパターンに起因する。本設計書は、ViteとReact 18の最新のエコシステム、Web Workerを前提としたオフスクリーン描画パイプライン、純粋関数に基づく幾何演算、そしてAntigravity IDEのエージェント型AI支援(Skills)を組み合わせることで、これらの問題を抜本的かつ完全に解決するための包括的なアーキテクチャとベストプラクティスを提示する。

課題1: Vite + React 18環境における PDF.js の安定運用

React 18において導入されたStrict Modeは、コンポーネントのマウントとアンマウントを意図的に繰り返すことで、副作用(Side Effects)のクリーンアップ漏れを開発時に顕在化させる。PDF.js(pdfjs-dist)は内部でCanvasの2DコンテキストやWeb Workerとの非同期通信を多用するため、このライフサイクルと極めて相性が悪い。非同期にページの取得やレンダリングが行われている最中にユーザーがページを切り替えた、あるいはコンポーネントがアンマウントされた場合、即座にタスクをキャンセルしないと未処理のPromise例外が発生し、メモリリークやアプリケーションのクラッシュを引き起こす¹。

技術的結論とベストプラクティス

PDF.jsの安定稼働には、単一の静的Workerの登録、useRefを用いたレンダリングタスク(RenderTask)の参照保持、そして一意な世代管理ID(renderRequestId)を用いた排他制御の3要素が不可欠である。レンダリングをキャンセルする際は、renderTask.cancel()を呼び出し、発生するRenderingCancelledExceptionを安全に捕捉して無視するパイプラインを構築する必要がある¹。さらに、iOSやChromeなどの環境において、巨大なPDFを開いた際に発生するメモリリークを防ぐため、PDFDocumentProxyのdestroy()メソッドをコンポーネントのアンマウント時に確実に実行しなければならない³。

Vite環境においては、動的インポートやURL解決を利用してWorkerファイルを正しく読み込む構成が求められる。これにより、モジュールバンドラーの制約を回避しつつ、Web Worker上での安全なPDFパースが可能となる²。

TypeScript実装コード片 (純粋ロジック層)

TypeScript

```
import { useEffect, useRef } from 'react'
import { pdfjsLib } from 'pdfjs-dist'
import type { PDFDocumentProxy, RenderTask } from 'pdfjs-dist'
```

```
// Vite環境における安全なWorker設定
```

```
pdfjsLib.GlobalWorkerOptions.workerSrc = new URL(
  'pdfjs-dist/build/pdf.worker.mjs',
  import.meta.url
).toString()
```

```
export function usePdfRenderer(pdfDataUrl: string, pageNumber: number)
```

```
  const canvasRef = useRef<HTMLCanvasElement> null
  const pdfRef = useRef<PDFDocumentProxy> null
  const renderTaskRef = useRef<RenderTask> null
  const renderRequestIdRef = useRef<number> 0
```

```
  useEffect(() => {
    let isSubscribed = true
```

```
    const loadAndRenderPdf = async () => {
      try {
        if (!pdfRef.current) {
          const loadingTask = pdfjsLib.getDocument(pdfDataUrl)
          pdfRef.current = await loadingTask.promise
        }

```

```
        if (!isSubscribed) return
```

```
        const pdf = pdfRef.current
        const page = await pdf.getPage(pageNumber)
```

```
        if (!isSubscribed) return
```

```
        // Retinaディスプレイ等に対応するためスケールを調整
        const viewport = page.getViewport({ scale: window.devicePixelRatio || 2.0 })
        const canvas = canvasRef.current
        if (!canvas) return
```

```
        const ctx = canvas.getContext('2d')
```

```
if (ctx) return
```

```
canvas.width = viewport.width
```

```
canvas.height = viewport.height
```

```
// 既存のレンダリングタスクを安全にキャンセル
```

```
if (renderTaskRef.current) {
```

```
  await renderTaskRef.current.cancel()
```

```
const requestId = ++renderRequestIdRef.current
```

```
const renderContext =
```

```
  canvasContext: ctx,
```

```
  viewport: viewport,
```

```
  |
```

```
const renderTask = page.render(renderContext)
```

```
renderTaskRef.current = renderTask
```

```
// レンダリングの完了を待機
```

```
await renderTask.promise
```

```
if (renderRequestIdRef.current === requestId) {
```

```
  // レンダリングが現在のリクエストに属している場合のみ完了処理を実行
```

```
  renderTaskRef.current = null
```

```
  catch (error: any) {
```

```
    // 意図的なキャンセルによる例外は握り潰さず安全にハンドリングする
```

```
    if (error?.name === 'RenderingCancelledException') {
```

```
      console.debug('PDF rendering cancelled safely.')
```

```
    } else {
```

```
      console.error('PDF rendering failed:', error)
```

```
    }
```

```
  }
```

```
  }
```

```
loadAndRenderPdf()
```

```
return () =>
```

```
  isSubscribed = false
```

```
  if (renderTaskRef.current) {
```

```
    renderTaskRef.current.cancel()
```

```

renderTaskRef.current = null
}

}

} Pdf(DataUrl, pageNumber))

useEffect(() => {
  // コンポーネントの完全なアンマウント時にドキュメントを破棄してメモリ解放
  return () => {
    if (pdfRef.current) {
      pdfRef.current.destroy()
      pdfRef.current = null
    }
  }
})

return canvasRef
}

```

避けるべきアンチパターン

コンポーネント内で直接pdfjsLib.getDocument()を呼び出し、その結果を単純なuseStateで管理する実装は避けるべきである。Reactのレンダリングサイクルと非同期処理のタイミングがずれることで、古いタスクが新しいタスクを上書きし、画面が点滅したりフリーズしたりする原因となる。また、error.name === "RenderingCancelledException"のハンドリングを省略すると、ブラウザのコンソールに未処理のPromise拒否 (Unhandled Promise Rejection) が溢れ、Viteの開発サーバーがクラッシュする危険性が高い。

課題2: A3巨大スキャン画像のメインスレッド凍結防止とメモリ管理

A3サイズ(300 DPIスキャン)の画像は、ピクセル数にしておよそ3500×5000ピクセル(約1750万画素)に達し、RGBAの非圧縮メモリ上では1枚あたり約70MBを占有する。これをブラウザのメインスレッド上のCanvasAPIで処理しようとする、黒枠検出や二値化のピクセル走査(getImageData)でUIスレッドが完全に数秒間停止(凍結)し、ユーザー体験を著しく損なう。さらに、V8エンジンのガベージコレクション(GC)が追いつかず、タブのメモリ不足クラッシュを誘発する⁶。

技術的結論とベストプラクティス

この問題の解決には、OffscreenCanvasを用いたバックグラウンドスレッド(Web Worker)へのピクセル演算の委譲と、ゼロコピー転送可能なImageBitmapの利用が必須となる⁷。メインスレッドで取得した元画像からcreateImageBitmap()によって非同期的かつハードウェア支援付きのデコードを行い、その参照をWorkerへpostMessageで送信する⁶。

Worker内では、大津の二値化(Otsu's Method)のアルゴリズムを利用して適応的な閾値を算出し、楽譜の有効領域(黒枠)を自動検出する⁹。大津のアルゴリズムは、前景と背景の画素値のクラス間分散(Between-class Variance)を最大化する閾値を自動的に決定するため、変色した楽譜や不均

一な照明下での判定において極めて堅牢である⁹。処理完了後は、不要となったImageBitmapに対して直ちに.close()を呼び出し、ガベージコレクションの実行を待たずにGPUバッファを強制解放することが、連続処理におけるメモリ断片化とクラッシュを防ぐ絶対的な要件である⁶。

TypeScript実装コード片 (純粋ロジック層)

TypeScript

```
// --- Worker内で実行される黒枠検出・大津の二値化ロジック (otsu.worker.ts) ---
```

```
self.onmessage = async (e: MessageEvent) =>
```

```
const { imageBitmap, action } = e.data
```

```
if (action !== 'PROCESS_IMAGE') return
```

```
// 計算負荷を下げるため、ダウンスケールしたOffscreenCanvasを構築
```

```
const TARGET_WIDTH = 800
```

```
const scale = TARGET_WIDTH / imageBitmap.width
```

```
const TARGET_HEIGHT = Math.floor(imageBitmap.height * scale)
```

```
const offscreen = new OffscreenCanvas(TARGET_WIDTH, TARGET_HEIGHT)
```

```
const ctx = offscreen.getContext('2d', { willReadFrequently: true })
```

```
if (!ctx) return
```

```
ctx.drawImage(imageBitmap, 0, 0, TARGET_WIDTH, TARGET_HEIGHT)
```

```
// メモリリーク防止のため明示的に元バッファを解放
```

```
imageBitmap.close()
```

```
const imageData = ctx.getImageData(0, 0, TARGET_WIDTH, TARGET_HEIGHT)
```

```
const data = imageData.data
```

```
const histogram = new Array(256).fill(0)
```

```
// 1. グレースケール化とヒストグラム作成
```

```
for (let i = 0; i < data.length; i += 4)
```

```
// 輝度計算 (Luminance: NTSC式)
```

```
const gray = Math.round(data[i] * 0.299 + data[i + 1] * 0.587 + data[i + 2] * 0.114)
```

```
histogram[gray]++
```

```
data[i] = data[i + 1] = data[i + 2] = gray
```

```
// 2. 大津の二値化アルゴリズムによる最適閾値の算出
```

```

const total = TARGET_WIDTH * TARGET_HEIGHT
let sum = 0
for (let i = 0; i < 256; i++) sum += i * histogram[i]

let sumB = 0; wB = 0; wF = 0; maxVariance = 0; threshold = 0

for (let i = 0; i < 256; i++) {
  wB += histogram[i]
  if (wB === 0) continue
  wF = total - wB
  if (wF === 0) break

  sumB += i * histogram[i]
  const mB = sumB / wB
  const mF = (sum - sumB) / wF
  const varianceBetween = wB * wF * (mB - mF) * (mB - mF)

  if (varianceBetween > maxVariance) {
    maxVariance = varianceBetween
    threshold = i
  }
}

```

```

// 3. バウンディングボックス(黒枠)の検出
let minX = TARGET_WIDTH, minY = TARGET_HEIGHT, maxX = 0, maxY = 0
for (let y = 0; y < TARGET_HEIGHT; y++) {
  for (let x = 0; x < TARGET_WIDTH; x++) {
    const idx = (y * TARGET_WIDTH + x) * 4
    // 閾値より暗いピクセルを楽譜(インク)の領域として判定
    if (data[idx] < threshold) {
      if (x < minX) minX = x
      if (x > maxX) maxX = x
      if (y < minY) minY = y
      if (y > maxY) maxY = y
    }
  }
}

```

```

// 4. 元画像のスケールに依存しないNormalized座標系(0.0 - 1.0)として結果を返す
const normalizedBBox = {
  x: minX / TARGET_WIDTH,
  y: minY / TARGET_HEIGHT,
  width: (maxX - minX) / TARGET_WIDTH,
}

```

```
height (maxY - minY) / TARGET_HEIGHT
}

self.postMessage(type 'SUCCESS' bbox normalizedBbox)
}
```

避けるべきアンチパターン

巨大な画像のピクセルデータをメインスレッド上で直接ループ処理(forループによる数百万回のイテレーション)することは、絶対に避けなければならない。また、CanvasからtoDataURL()を使用してBase64文字列に変換する処理は、メモリ使用量を元バイナリの約1.37倍に膨張させ、GCの処理を停止させてアプリケーションをクラッシュさせる最大要因となる。処理対象のデータは常に転送可能オブジェクト(Transferable Objects)としてスレッド間を移動させるべきである。

課題3: 幾何座標系 (Geometry) の統一と純粋関数アーキテクチャ

ブラウザ完結型の画像編集エディタにおいて、クロップ位置のズレや拡大縮小時の破綻が生じる主要な原因は、性質の異なる3つの座標系が同じ状態管理(State)内で混同されることにある。PythonやOpenCVを用いたバックエンド処理では画像マトリクス(Source座標)のみを扱えばよいが、フロントエンドではブラウザのレイアウトシステムが複雑に絡み合う。

技術的結論とベストプラクティス

アーキテクチャの根幹として、状態管理(ReduxやReact Context)には必ずNormalized座標(0.0～1.0の浮動小数点)のみを保存する。ビューへのレンダリングやエクスポートを行う直前の末端層においてのみ、純粋関数を用いて各座標系へ変換を行う。これにより、デバイスの画面幅が変わったり、DPIの異なる画像を読み込んだりしても、クロップ位置や分割線が完全に同一の場所を指し示すことが保証される。見開きA3からA4二枚への分割処理も、このNormalized領域の幅を半分にする(width / 2)という純粋な数学的演算として簡潔に実装でき、Python版の幾何ロジックをTypeScriptへ安全に移植することが可能となる。

座標系	単位	用途と特徴	状態管理での永続化
Normalized	0.0 ~ 1.0	アプリケーションの単一の真実の情報源。解像度に依存しない幾何情報の管理。	保持する(必須)
Screen	CSS Pixels	ブラウザ上でのUIレンダリング、DOM要素の描画、ドラッグイベントなどのハンドリ	保持しない(都度計算)

		ング。	
Source	物理Pixels	最終的なPDF・画像エクスポート、高解像度クロップ処理、PDF.jsによるレンダリング。	保持しない(都度計算)

TypeScript実装コード片 (純粋ロジック層)

TypeScript

```

export interface Point { x: number; y: number; }
export interface Rect { x: number; y: number; width: number; height: number; }
export interface Size { width: number; height: number; }

export const GeometryConverter = {
  /**
   * UI上のドラッグ位置 (Screen座標) を状態管理用のNormalized座標に変換する。
   * @param point ドラッグされたScreen座標
   * @param containerSize 画像を表示しているコンテナのScreenサイズ
   */
  screenToNormalized (point: Point, containerSize: Size): Point => {
    return {
      x: Math.max(0, Math.min(1, point.x / containerSize.width)),
      y: Math.max(0, Math.min(1, point.y / containerSize.height))
    };
  },

  /**
   * Normalized座標をエクスポート用のSource座標 (実ピクセル) に変換する。
   * この関数によりサブピクセルのズレを丸め込み、正確なピクセルマッピングを行う。
   */
  normalizedToSourceRect (normalizedRect: Rect, sourceSize: Size): Rect => {
    return {
      x: Math.round(normalizedRect.x * sourceSize.width),
      y: Math.round(normalizedRect.y * sourceSize.height),
      width: Math.round(normalizedRect.width * sourceSize.width),
      height: Math.round(normalizedRect.height * sourceSize.height)
    };
  }
};

```



```

/**
 * A3見開きなどのクロップ領域を、正確に左右2枚のA4(Normalized)に分割する純粋関数。
 * Python版のロジックと完全に等価であり、解像度に依存しない。
 */
splitNormalizedRectIntoTwo (rect: Rect): [Rect, Rect] =>
  const halfWidth = rect.width / 2
  return
    [
      { x: rect.x, y: rect.y, width: halfWidth, height: rect.height }, // 左ページ
      { x: rect.x + halfWidth, y: rect.y, width: halfWidth, height: rect.height } // 右ページ
    ]

```

避けるべきアンチパターン

ドラッグイベントの座標(e.clientXやe.clientY)をそのままピクセル値としてStateに保存し、ウィンドウのリサイズ時やデバイス回転時に状態を再計算するアーキテクチャは破綻の元である。また、CSSのtransform: scale()とJavaScript上の座標演算を同時に行うと、ブラウザの実装差異により数ピクセルの小数点のズレ(サブピクセルレンダリングの不一致)が生じ、最終的なクロップ境界に黒い線やノイズが残る原因となる。

課題4: 300 DPI 無劣化 PDF 出力パイプライン (pdf-lib)

Score Optimizer 2.0の最終生成物は、オーケストラの印刷用途に耐えうる300 DPIの無劣化A4 PDFである。pdf-libを利用する場合、PDFの内部座標系はDTPの標準であるポイント(pt)で計算され、A4サイズの寸法は正確に 595.28 x 841.89 ポイントとして定義される¹³。PDFの標準仕様では1pt = 1/72インチとして扱われるため、ピクセルベースの画像を無劣化で配置するには、アフィン変換を用いた精密なスケーリング行列の算出が不可欠である¹³。

技術的結論とベストプラクティス

スキャン画像の高解像度バイト配列(Blob)を再エンコードすることなく、直接embedJpgまたはembedPngを利用してPDFドキュメントに埋め込むことで、無劣化出力を実現する。用紙サイズをA4([595.28, 841.89])に設定し、画像のアスペクト比を維持しながら用紙の中央に配置するためのスケーリング係数(scaleX, scaleY)と移動量(tx, ty)を数学的に算出する¹³。

さらに、最終的なファイルのダウンロードにおいて、生成されたBlobを適切にハンドリングし、URLオブジェクトを直ちにrevokeすることで、出力パイプラインにおけるメモリリークを完全に遮断する。revokeのタイミングが早すぎるとダウンロードが失敗するため、requestAnimationFrameを挟むことでブラウザの次フレームでの安全な破棄を保証する。

TypeScript実装コード片 (純粋ロジック層)

TypeScript

```
import { PDFDocument } from 'pdf-lib'

export async function createA4PdfFromImageBlobs(imageBlobs: Blob[]): Promise<Blob> {
  // A4サイズの定義 (1pt = 1/72 inch).
  // pdf-libの仕様に準拠した定数
  const A4_WIDTH = 595.28
  const A4_HEIGHT = 841.89

  const pdfDoc = await PDFDocument.create()

  for (const blob of imageBlobs) {
    const arrayBuffer = await blob.arrayBuffer()
    let pdfImage

    // 画像フォーマットに応じて再エンコードなしで無劣化埋め込み
    if (blob.type === 'image/jpeg') {
      pdfImage = await pdfDoc.embedJpg(arrayBuffer)
    } else if (blob.type === 'image/png') {
      pdfImage = await pdfDoc.embedPng(arrayBuffer)
    } else {
      throw new Error 'Unsupported image format. Use JPEG or PNG.'
    }

    const page = pdfDoc.addPage([A4_WIDTH, A4_HEIGHT])

    // 画像本来のピクセルサイズを取得
    const imageDims = pdfImage.scale(1)

    // Fit-to-page センタリングのスケール係数計算 (sx, sy)
    const scaleX = A4_WIDTH / imageDims.width
    const scaleY = A4_HEIGHT / imageDims.height
    const scale = Math.min(scaleX, scaleY) // アスペクト比を維持して収める

    const scaledWidth = imageDims.width * scale
    const scaledHeight = imageDims.height * scale

    // 中央配置のための並進オフセット計算 (tx, ty)
    const xOffset = (A4_WIDTH - scaledWidth) / 2
    const yOffset = (A4_HEIGHT - scaledHeight) / 2

    page.drawImage(pdfImage,
```

```

    x: xOffset,
    y: yOffset,
    width: scaledWidth,
    height: scaledHeight,
  },
)
}

const pdfBytes = await pdfDoc.save()
return new Blob([pdfBytes], { type: 'application/pdf' })
}

export function downloadSafeBlob(blob: Blob, filename: string): void {
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a')
  a.href = url
  a.download = filename
  document.body.appendChild(a)
  a.click()

  // GC(ガベージコレクション)にメモリを解放させるためのクリーンアップ
  // ダウンロード処理の開始を妨害しないよう requestAnimationFrame で遅延評価する
  requestAnimationFrame(() => {
    document.body.removeChild(a)
    URL.revokeObjectURL(url)
  })
}

```

避けるべきアンチパターン

画像を一旦Canvasに描画し、そこからtoDataURL('image/jpeg', 1.0)を生成してpdf-libに渡すフローは最悪のアンチパターンである。これは無駄な再エンコードを強制するだけでなく、デバイスのDPI設定(Retinaディスプレイ等のピクセル比)に影響され、最終出力の解像度が低下する(またはファイルサイズが異常に肥大化する)原因となる。元のBlobバイト配列を直接PDFのオブジェクトストリームに埋め込む(embed)ことが唯一の正解である。

課題5: Apple HIG 準拠のレスポンス UI/UXアーキテクチャ

オーケストラの現場では、iPad(iOS/iPadOS)を用いてブラウザから直接スキャンと編集を行うケースが多発する。この際、Safariの動的なアドレスバーや下部のホームインジケーターがUIと干渉し、ボタンが押せなくなったり、クロップ枠のハンドルが画面外に消えたりする現象が生じる。加えて、指によるタッチ操作はマウスに比べて精度が低く、クロップ境界の微調整が困難となる。

技術的結論とベストプラクティス

レイアウトの基盤として、旧来の100vhを完全に廃止し、動的ビューポート単位である100dvhを採用する。加えて、Apple Human Interface Guidelines (HIG) に準拠するため、全てのタッチインター

フェースは最低 44x44pt のタップ領域を確保する。クロップ用の8点ハンドル(四隅+四辺)は、視覚的な要素(例: 12x12pxの丸)と、透明でより大きなタッチターゲット(44x44px)を分離するネスト構造(コンポーネント構造)で設計しなければならない。

さらに、精密なクロップを補助するため、ドラッグ中の領域を拡大表示する「ズームHUD(Loupe)」を画面の安全な領域(Safe Area)にフロート表示させる。これにより、ユーザーは指で隠れたクロップ境界線をサブピクセル単位で正確に視認することが可能となる。

TypeScript / CSS 実装コード片 (純粋ロジック層)

CSS

```
/* CSSアーキテクチャ: Safe AreaとDynamic Viewportの統合 */
```

```
.app-container
```

```
/* Safariの動的アドレスバーに対応するdvhを使用 */
```

```
height 100dvh
```

```
width 100vw
```

```
overflow hidden
```

```
/* HIG準拠のセーフエリア制約 */
```

```
padding-bottom env(safe-area-inset-bottom)
```

```
padding-top env(safe-area-inset-top)
```

```
display flex
```

```
flex-direction column
```

```
|
```

```
/* Apple HIG準拠のクロップハンドル構造 */
```

```
.crop-handle-touch-target
```

```
position absolute
```

```
width 44px /* 最小タップターゲットの保証 */
```

```
height 44px
```

```
transform translate(50%, 50%) /* 中心を頂点に合わせる */
```

```
cursor crosshair
```

```
/* タッチターゲット自体は透明にする */
```

```
background transparent
```

```
display flex
```

```
align-items center
```

```
justify-content center
```

```
|
```

```
.crop-handle-visual
```

```
width 12px
```

```
height 12px
```

```
background-color #007AFF /* iOS Blue */
border-radius 50%
border 2px solid white
box-shadow 0 1px 3px rgba 0 0 0 0.3

```

```
/* ズームHUD (Loupe) のレイアウト */
.zoom-hud-container
  position absolute
  top env(safe-area-inset-top) 16px
  left 16px
  width 120px
  height 120px
  border-radius 50%
  overflow hidden
  border 4px solid #fff
  box-shadow 0 4px 12px rgba 0 0 0 0.5
  pointer-events none

```

TypeScript

```
// ズームHUDの背景位置を算出する純粋関数
export const calculateZoomHudBackgroundPosition =
  (normalizedDragPoint: Point,
   sourceImageSize: Size,
   hudSize: number = 120,
   zoomScale: number = 2.0,
   string =>
    const pixelX = normalizedDragPoint.x * sourceImageSize.width
    const pixelY = normalizedDragPoint.y * sourceImageSize.height

    // ズーム対象の中心点がHUDの中央にくるように背景オフセットを計算
    const bgPosX = -(pixelX * zoomScale) + (hudSize / 2)
    const bgPosY = -(pixelY * zoomScale) + (hudSize / 2)

    return `${bgPosX}px ${bgPosY}px`

```

避けるべきアンチパターン

視覚的なハンドル要素(例えばwidth: 10px; height: 10px;)に直接onClickやonTouchStartをバインドすることは絶対に行わない。モバイル環境において指先で正確に10pxの要素を捉えることは不可能であり、ユーザーのフラストレーションを極限まで高める結果となる。また、固定値のvhを使用したままpaddingで誤魔化すレイアウトは、デバイスの向き(ポートレート・ランドスケープ)が変更された瞬間に確実に破綻する。

課題6: Antigravity IDE の Skill 調査と導入提案

Score Optimizer 2.0のように、Reactのライフサイクル、Web Workerのメモリ管理、幾何学的な純粋関数、そしてバイナリ操作が複雑に絡み合う高度なプロジェクトでは、開発者(またはAIコーディングアシスタント)がすべてのコンテキストと過去のアンチパターンを暗記してコードを記述・修正することは不可能に近い。Googleが提供するエージェント型開発プラットフォーム「Antigravity IDE」の最大の特徴である「Skills(スキル)」機能は、まさにこのような属人的なドメイン知識をAIエージェントにハードコードし、自律的な専門家として振る舞わせるためのアーキテクチャである¹⁸。

Antigravity IDEのSkillの機能とアーキテクチャ

Skillは、単なるプロンプトテンプレートではない。AIに環境内でコードを実行する権限と手順を付与する、ディレクトリベースのモジュールパッケージである²¹。最も重要な概念は「段階的開示(Progressive Disclosure)」であり、通常時は軽量のメタデータのみをメモリに保持し、AIがユーザーの要求をトリガーとして判断した際のみ、重いドキュメントや実行スクリプト(PythonやBashなど)をコンテキストに読み込む¹⁹。これにより、数十万トークンを消費する「コンテキストの飽和(Context Saturation)」やモデルの混乱を防ぎ、大規模プロジェクトでも高精度な推論を維持する¹⁹。

Skillの構造は主に以下のファイルとディレクトリから成る¹⁸:

1. **SKILL.md**: スキルの「脳」となる定義ファイル。YAMLフロントマター(name, description)とMarkdownの本文(Goal, Instructions, Constraints)で構成される。
2. **scripts/**: PythonやNode.jsなどの実行スクリプト。エージェントがシェルを介して自律的に実行するツール。
3. **resources/**: チェックリスト、アーキテクチャの定義書、参考資料。
4. **examples/**: 理想的なコードや入出力のFew-shotサンプル。

スキルのスコープ	保存ディレクトリ	適用範囲と用途
Global Scope	~/gemini/config/skills/	全プロジェクト共通。一般的なコードフォーマットや汎用ユーティリティ(例: frontend-design, ui-ux-pro-max) ¹⁸ 。
Workspace Scope	.agent/skills/	プロジェクト固有。特定のデータベース管理、フレームワークのボイラープレート、独自のコーディング規約の徹底

本プロジェクトへの導入提案（ベストプラクティス）

Score Optimizer 2.0の開発効率とコード品質を最大化するために、既存の強力なグローバルスキルを活用しつつ、プロジェクト特有のWorkspaceスコープのカスタムスキルを構築・導入するアプローチを提案する。

1. 導入すべき既存のグローバルスキル

Antigravityのエコシステムには多数のスキルが存在するが、プロジェクトに無関係なものを大量にインストールするとIDEのクラッシュやツールの枯渇を招くため、厳選が必要である²³。

- **frontend-design**: UI/UXの構築において、適切な余白、タイポグラフィ、一貫性のあるデザインをAIに生成させるために必須のスキル。バニラなAI出力とは一線を画す品質を提供する²³。
- **ui-ux-pro-max**: ダッシュボードやユーザー向けのオンボーディングフローなど、マークアップだけでなくユーザー体験そのものをAIに考慮させる場合に有効である²³。

2. 本プロジェクト固有のカスタムスキル構築

本プロジェクトの複雑な制約をAIに遵守させるため、プロジェクトのルートディレクトリ(.agent/skills/)に以下の3つの特化型Skillを自作して配置する¹⁸。

1. pdf-lifecycle-reviewer (PDFメモリーリーク自動検知スキル)

- 目的: Reactコンポーネントが生成・変更された際、エージェントが自動的にuseEffect内のクリーンアップ処理を静的解析し、pdfjs-distのdestroy()やcancel()が欠落していないかを自律的にチェックする。

2. geometry-guard (座標系混同防止スキル)

- 目的: クロップやUI操作に関するロジックを実装する際、AIに対して「状態管理は必ずNormalized座標(0.0 - 1.0)で行う」という本プロジェクトのアーキテクチャルールを強制する¹⁹。examples/ フォルダに GeometryConverter のお手本コードを配置し、学習テンプレートとして利用させる。

3. otsu-worker-tester (オフスクリーン処理・テスト自動化スキル)

- 目的: メインスレッドの凍結を防ぐため、画像処理を伴う機能追加時には、AI自身にダミーのWorkerスクリプトを生成させ、OffscreenCanvasを用いたメモリプロファイル(close())の呼び出し確認を含む)のテストを実行させる⁶。

カスタムスキルの実装例 (.agent/skills/pdf-lifecycle-reviewer/SKILL.md)

YAML

name: pdf-lifecycle-reviewer

description: Reactコンポーネントのコード変更時に呼び出される。PDF.jsの初期化・破棄手順や、メモリーリーク(destroy, cancelの欠落)を防ぐためのレビューと修正を自律的に行う。

Goal

React 18のStrict Modelにおけるコンポーネントのマウント/アンマウント時において、PDF.jsの

RenderingCancelledExceptionの放置や、Web Workerのメモリリークを完全に防ぐ。

Instructions

1. 対象のReactコードをスキャンし、`pdfjsLib.getDocument` の呼び出し箇所を特定する。
2. `useEffect` のクリーンアップ関数(return文)において、非同期タスクに対する `cancel()` および `PDFDocumentProxy` に対する `destroy()` が記述されているか確認する。
3. 欠落がある場合、`resources/pdf-best-practice.md` に従って、`useRef` を用いた状態管理へコードをリファクタリングし、差分(Diff)を提案・適用する。

Constraints

- メインスレッド上で巨大なピクセル配列(ImageData)を直接ループ処理するコードを生成してはならない。必ずWeb WorkerとOffscreenCanvasを利用すること。
- エラーハンドリングにおいて `RenderingCancelledException` を握り潰すのではなく、明示的に無視するロジック(`if (error.name === 'RenderingCancelledException')`)を記述すること。

避けるべきアンチパターン

「awesome-skills」リポジトリなどから何十もの不必要なスキルを盲目的にバルクインストールすることは絶対に避けるべきである。ツールのロード制限に引っかかり、IDE自体がクラッシュする原因となる²³。また、全ての社内コーディング規約を1つの巨大なプロンプト(システムプロンプトやCustomizations)に詰め込むことも避けるべきである。これはコンテキストの腐敗(Context Rot)を引き起こし、AIが本当に必要なアーキテクチャの要件を忘却する原因となる¹⁹。Skillは単一責任の原則に従い、目的ごとに細分化してディレクトリを分けるべきである。

結論

見開き楽譜スキャンPDFをブラウザ上で安全かつ無劣化で最適化する「Score Optimizer 2.0」の要件は、従来の単純なSPA(Single Page Application)の枠を超え、高度なブラウザ内部のメモリ管理やマルチスレッドプログラミングの領域に達している。

React 18のライフサイクルとPDF.jsの競合は、レンダリングタスクの厳密な参照管理と例外の安全な捕捉によって完全に制御可能である。また、A3巨大スキャンのメインスレッド凍結問題は、OffscreenCanvasとImageBitmapを用いたWorkerへのオフロードと、大津の二値化アルゴリズムによる自律的閾値計算によって解決される。幾何処理における純粋関数の導入は、デバイスの解像度に依存しない完璧なクロップを保証し、pdf-libを用いた無劣化アフィン変換による再構成が、最終的な300 DPI印刷品質を担保する。

最後に、これらの高度なアーキテクチャ決定と制約事項をチームやAIエージェントに継続的に遵守させるため、Antigravity IDEの「Skills」機能を導入する。段階的開示アーキテクチャに基づく特化型Skill群(PDFメモリ管理、Geometry制約、Worker自動テスト)をプロジェクトルートに配置することで、AIは単なるコードジェネレータから、プロジェクトのアーキテクチャを熟知し、アンチパターンを自律的に排除する熟練のアーキテクトへと進化する。この一連の技術スタックと運用基盤の統合こそが、1ミリの狂いもない高精度なWebアプリケーションを本番環境で長期にわたり安定稼働させるための唯一の道である。

引用文献

1. RenderingCancelledException: Rendering cancelled, page 1 · Issue, <https://github.com/FranckFreiburger/vue-pdf/issues/202>
2. React PDF viewer with pdfjs-dist and Next.js (2026) - Nutrient, <https://www.nutrient.io/blog/how-to-build-a-reactjs-viewer-with-pdfjs/>
3. ng2-pdf-viewer - UNPKG, <https://app.unpkg.com/ng2-pdf-viewer@6.1.0/files/CHANGELOG.md>
4. CHANGELOG.md - ng2-pdf-viewer - GitHub, <https://github.com/VadimDez/ng2-pdf-viewer/blob/master/CHANGELOG.md>
5. Comparing the best React PDF viewers for developers - Nutrient, <https://www.nutrient.io/blog/top-react-pdf-viewers/>
6. [Proposal] ImageBitmap.getImageData #4785 - whatwg/html - GitHub, <https://github.com/whatwg/html/issues/4785>
7. Web Workers - 33 JavaScript Concepts, <https://33jsconcepts.com/concepts/web-workers>
8. Web Workers and Transferable Objects: Off-Main-Thread ... - Anirone, <https://anirone.com/blog/general/web-workers-transferable-objects>
9. Image thresholding with Otsu method | Hui Wang's Blog, <https://xydida.com/2022/10/2/ComputerVision/Image-thresholding-with-Otsu-method/>
10. Otsu's method for image thresholding explained and implemented, <https://muthu.co/otsus-method-for-image-thresholding-explained-and-implemented/>
11. Convert an Image to Black and White with JavaScript - Dynamsoft, <https://www.dynamsoft.com/codepool/convert-image-to-black-white-with-javascript.html>
12. memory leak in JavaScript (WebWorker, Canvas, IndexedDB), <https://stackoverflow.com/questions/67148570/memory-leak-in-javascript-webworker-canvas-indexeddb>
13. Introduction - PDF-LIB, <https://pdf-lib.js.org/docs/api/>
14. size of page (ex. A4) · Issue #143 · Hopding/pdf-lib - GitHub, <https://github.com/Hopding/pdf-lib/issues/143>
15. Document what the unit is for page dimensions · Issue #1531 - GitHub, <https://github.com/Hopding/pdf-lib/issues/1531>
16. PDF-LIBでカレンダーを作れるサイトを作ったので - Zenn, https://zenn.dev/stafes_blog/articles/shota1995m-calendar-pdf
17. Node.js + pdf-libの使い方 (用紙サイズ、向き、文字出力、改ページ), <https://symfoware.blog.fc2.com/blog-entry-2598.html>
18. Authoring Google Antigravity Skills - Codelabs, <https://codelabs.developers.google.com/getting-started-with-antigravity-skills>
19. Google Antigravity Skills - AiSpirits, <https://aispirits.com/2026/01/27/google-antigravity-skills/>
20. 【イラスト付きやさしい解説】Antigravityでskills導入しよう！なぜ, https://note.com/vast_gnu572/n/n0613f591a78e
21. antigravity-skill-creator/SKILL.md at main - GitHub, <https://github.com/mohan25manu/antigravity-skill-creator/blob/main/SKILL.md>

22. 【Google Antigravity】新機能「Skills」について - Zenn,
https://zenn.dev/emp_tech_blog/articles/google-antigravity-skills
23. The Right Way to Use Antigravity IDE: Skills and MCP Servers That,
<https://medium.com/@huzaifa3108hassan/the-right-way-to-use-antigravity-ide-skills-and-mcp-servers-that-actually-matter-e26ed1767e0d>